

Penerapan Big Data Terhadap Transaksi Sales Pada Bisnis Toko Online Berbasis Di Inggris

- Tugas Besar Mata Kuliah Prinsip Ekonomi Mikro (SD3105) -

1st Barrera Putra
120450078

2nd Arafı Ramadhan Maulana
122450002

3rd Sесilia Putri Subandi
122450012

4th Febiya Jomy Pratiwi
122450074

5th Rian Bintang Wijaya
122450094

6th Anwar Muslim
122450117

7th Dinda Nababan
122450120

Abstrak

Dalam kemajuan teknologi informasi yang berbasis big data tentunya dapat digunakan sebagai pemanfaatan analisis big data dalam berbagai sektor kehidupan, termasuk sektor dan ruang lingkup ekonomi mikro. Pemanfaatan analisis big data dalam sektor ekonomi sangat banyak dan berdampak penting terutama bagi pelaku usaha untuk mengetahui perilaku konsumen dan gambaran umum pasar agar dapat membuat keputusan terkait harga dan produknya sehingga mencapai keuntungan maksimal dan alasan tersebut yang melandasi pelaksanaan dari penelitian ini. Metode yang akan digunakan pada penelitian ini adalah uji Spearman, uji Pearson, uji Kendall, segmentasi data dan analisis time-series serta pemanfaatan library pandas, numpy, random, matplotlib dan seaborn yang diterapkan untuk menguji korelasi harga dan kuantitas barang untuk mengetahui nilai elastisitas harga, segmentasi produk/barang serta tren toko dalam suatu acuan waktu hari/minggu/bulan.

Kata Kunci: Analisis Big Data, Ekonomi Mikro, Keputusan pelaku usaha

I. Pendahuluan

A. Latar Belakang

Dunia industri adalah bagian penting dari evolusi teknologi. Kemajuan teknologi yang terus berkembang memiliki dampak besar pada kehidupan masyarakat sepanjang berbagai era, mulai dari era teknologi pertanian, industri, hingga era teknologi informasi, dan era teknologi komunikasi dan informasi yang semakin meluas terhadap jaringan internet telah membawa transformasi digital yang signifikan. [1]

Dengan transformasi digital yang berkembang dengan cepat, masyarakat harus berubah untuk menggunakan dan menerapkan teknologi dalam kehidupan sehari-hari, termasuk bisnis. *E-Commerce*, atau sebuah platform yang memungkinkan penjualan berbagai produk secara *online*, merupakan salah satu peran dalam

kegiatan bisnis, yang secara signifikan meningkatkan kinerja bisnis. [2]

Dalam kegiatan bisnis mengelola transaksi penjualan merupakan bagian penting dalam menjalankan toko *online*. Dengan semakin banyaknya transaksi dan kompleksitas data yang dihasilkan oleh toko *online*, muncul tantangan baru dalam pengelolaan data. Oleh karena itu, diperlukan solusi yang efektif dan kreatif untuk mengoptimalkan proses perdagangan. Dalam situasi seperti ini, penggunaan teknologi *Big Data* tidak hanya menjadi relevan namun juga strategis.

B. Rumusan Masalah

Dengan transformasi digital yang pesat, terutama dalam sektor bisnis dan perdagangan elektronik (*E-Commerce*), mengelola transaksi penjualan menjadi sangat penting bagi pemilik toko *online*. Tantangan baru dalam pengelolaan data muncul seiring dengan jumlah transaksi yang meningkat dan kompleksitas data yang dihasilkan.

Dengan munculnya kondisi ini, solusi inovatif dan efisien diperlukan untuk mengoptimalkan proses perdagangan bisnis *E-Commerce*. Penggunaan teknologi *Big Data* mungkin merupakan salah satu solusi yang muncul. Meskipun *Big Data* sangat penting dalam konteks ini, pertanyaan penting tentang implementasi strategis dan efisien masih muncul. Oleh karena itu, masalah utama adalah bagaimana mengintegrasikan dan mengimplementasikan teknologi *Big Data* secara efektif dalam pengelolaan transaksi penjualan perusahaan *E-Commerce* untuk

meningkatkan kinerja dan efisiensi mereka.

C. Deskripsi Data

Pada kasus ini digunakan dataset yang diperoleh dari situs resmi *Kaggle* (<https://www.kaggle.com/code/ekajaya/analysis-dataset-sales-transaction-v-4a-csv>) berupa data *sales transaction* pada toko *online* berbasis di Inggris. Data tersebut akan diolah sedemikian rupa untuk menghasilkan analisis penerapan *Big Data* terhadap transaksi *sales* pada bisnis toko *online* berbasis di Inggris.

II. Rancangan Sistem

A. Gambaran Umum

Pengolahan data dilakukan dengan menggunakan *library* *pandas* untuk membaca file *csv* dari data set yang telah diunduh. Setelah itu, akan dilakukan pembersihan data agar data lebih siap untuk digunakan. Pada pembersihan data/*data cleansing* akan diperiksa *null values* dari data, kemudian akan diperiksa juga duplikasi dari data yang ingin digunakan. Setelah itu *null values* dan data duplikasi tersebut akan dibersihkan.

Setelah data siap digunakan, maka dilakukanlah pengolahan data agar data dapat dianalisis dengan mudah. Hasil dari pengolahan data tersebut diharapkan mampu menganalisis penerapan *Big Data* terhadap transaksi *sales* pada bisnis toko *online* berbasis di Inggris.

B. Library yang Digunakan

Pada kasus ini, terdapat beberapa *library* yang digunakan dalam pengolahan data. Beberapa *library* tersebut diantaranya ialah *library* pandas, numpy, random, matplotlib, dan seaborn.

Pandas merupakan *library* pada Python yang biasa digunakan untuk menganalisis data, seperti memanipulasi data dan pembersihan data. *Library* Pandas juga dibuat untuk mengolah kumpulan data besar secara cepat dan efisien. Sementara itu Numpy merupakan *library* Python yang fokus pada *scientific computing*. Numpy memiliki kemampuan untuk membentuk objek N-dimensional *array*, yang mirip dengan *list* pada Python [3].

Random merupakan sebuah *library* bawaan dari Python yang berfungsi untuk menampilkan suatu output secara acak [4].

Matplotlib adalah *library* Python 2 Dimensi yang dapat menghasilkan plot dengan kualitas tinggi dalam berbagai format dan dapat digunakan di banyak platform. Matplotlib dapat digunakan sebagai pembuat grafik dalam berbagai platform, seperti Python dan Jupyter Notebook. Grafik yang dapat dibuat oleh matplotlib cukup beragam, seperti grafik garis, batang, lingkaran, histogram dan berbagai macam lainnya. Sama halnya dengan matplotlib, seaborn juga merupakan sebuah *library* yang bertujuan untuk membuat grafik dan statistik dengan menggunakan Python. Seaborn dibangun berdasarkan *library* matplotlib serta terintegrasi dengan struktur data pada Pandas [5].

C. Daftar Fungsionalitas

Berikut merupakan beberapa daftar fungsionalitas dari pengolahan data penerapan *Big Data* terhadap transaksi sales pada bisnis toko online berbasis di Inggris.

1. `df_sales = pd.read_csv`, digunakan untuk membaca dataset berupa file excel berformat csv.
2. `df_sales`, digunakan untuk menampilkan/memanggil dataset
3. `df['Date'] = pd.to_datetime(df['Date'])`, digunakan untuk mengkonversi kolom 'Date' menjadi tipe data datetime
4. `df_cleaned = df.drop_duplicates()`, digunakan untuk menghapus/membersihkan data duplikat
5. `df_cleaned.to_csv('data_cleaned2.csv', index=False)`, digunakan untuk menyimpan data yang sudah bersih ke file csv.
6. `filtered_df = df[df['CustomerNo'].isnull()]`, digunakan untuk memfilter data yang memiliki nilai null.
7. `df.isnull().sum()`, digunakan untuk memeriksa/mencetak apakah data masih memiliki nilai null.

III. Hasil dan Pembahasan

Big Data memiliki banyak peran dalam segala aspek kehidupan, salah

satunya yaitu bidang ekonomi. Ilmu ekonomi sendiri terbagi atas dua jenis berdasarkan ruang lingkungannya, yaitu ekonomi mikro dan ekonomi makro. Dalam penelitian ini, dilakukan pengkajian secara literasi dan eksperimen dalam bentuk pengolahan data untuk mengetahui peran dan penerapan *big data* di bidang ekonomi mikro.

Penelitian ini secara spesifik membahas tentang penerapan *big data* terhadap transaksi *sales* pada toko *online* berbasis di Inggris. Dataset yang digunakan sebagai bahan penelitian ini berasal dari *website open source* Kaggle.

```
[ ] 1 df_sales = pd.read_csv('Sales_Transaction_v.4a.csv')
```

```
[ ] 1 df_sales
```

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: 'should_run_async' will not call 'tran
and should_run_async(code)
```

TransactionNo	Date	ProductNo	ProductName	Price	Quantity	CustomerNo	Country
0	581482	12/9/2019	22485	Set Of 2 Wooden Market Crates	21.47	12	17490.0 United Kingdom
1	581475	12/9/2019	22596	Christmas Star Wreath List Chalkboard	10.65	36	13069.0 United Kingdom
2	581475	12/9/2019	22325	Storage Tin Vintage Leaf	11.53	12	13069.0 United Kingdom
3	581475	12/9/2019	22372	Tree T-Light Holder Wattle Winkle	10.65	12	13069.0 United Kingdom
4	581475	12/9/2019	22329	Set Of 4 Knick Knack Tins Poppies	11.94	6	13069.0 United Kingdom
...	...	...	...	...	...	...	...
536345	C358548	12/1/2018	22168	Organiser Wood Antique White	18.96	-2	12472.0 Germany
536346	C358548	12/1/2018	21218	Red Spotty Biscuit Tin	14.09	-3	12472.0 Germany
536347	C358548	12/1/2018	20957	Porcelain Hanging Bell Small	11.74	-1	12472.0 Germany
536348	C358548	12/1/2018	22580	Advent Calendar Gingham Sack	16.35	-4	12472.0 Germany
536349	C358548	12/1/2018	22767	Triple Photo Frame Cornice	20.45	-2	12472.0 Germany

536350 rows x 8 columns

Gambar 3.1

Dataset diimport ke dalam google colab. Dataset ini terdiri dari 536350 baris dan 8 kolom yang berisi nomor transaksi, tanggal, nomor produk, nama produk, harga, kuantitas, nomor pelanggan dan negara seperti terlampir pada Gambar 3.1.

```
1 import pandas as pd
2
3 # Baca dataset CSV
4 df = pd.read_csv('Sales_Transaction_v.4a.csv')
5
6 # Konversi kolom 'Date' menjadi tipe data datetime
7 df['Date'] = pd.to_datetime(df['Date'])
8
9 # Temukan tanggal awal dan tanggal akhir
10 tanggal_awal = df['Date'].min()
11 tanggal_akhir = df['Date'].max()
12
13 # Cetak hasilnya
14 print(f'Rentang waktu data dalam dataset: dari {tanggal_awal} hingga {tanggal_akhir}")
```

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: 'should_run_async' will not ca
and should_run_async(code)
```

Rentang waktu data dalam dataset: dari 2018-12-01 00:00:00 hingga 2019-12-09 00:00:00

Gambar 3.2

Setelah mengimport dataset, dilakukan pembacaan dataset yang berbentuk csv. Dilakukan konversi data

kolom tanggal menjadi tipe data *datetime* untuk mengetahui rentang waktu dari dataset. didapatkan rentang waktu dataset yang digunakan yaitu dari 2018-12-01 00:00:00 hingga 2019-12-09 00:00:00 (1 tahun 8 hari).

• Memeriksa nilai null values

```
1 import pandas as pd
2
3 # Baca data dari file CSV
4 df = pd.read_csv('Sales_Transaction_v.4a_cleaned.csv')
5
6 # Cetak jumlah nilai null
7 print(df.isnull().sum())
```

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: 'should_run_async'
and should_run_async(code)
```

TransactionNo	0
Date	0
ProductNo	0
ProductName	0
Price	0
Quantity	0
CustomerNo	1
Country	0
dtype:	int64

Gambar 3.3

Demi mengoptimalkan proses analisis dengan jumlah data yang besar, dilakukan *data cleansing*. Pertama yaitu memeriksa nilai *null* dari dataset, didapatkan 1 nilai *null* pada kolom nomor pelanggan seperti terlampir pada Gambar 3.3.

• Memeriksa duplikasi dan menghilangkan duplikasi data

```
1 import pandas as pd
2
3 # Baca data dari file CSV
4 df = pd.read_csv('Sales_Transaction_v.4a_cleaned.csv')
5
6 # Menampilkan jumlah data duplikat sebelum membersihkan
7 jumlah_duplikat_sebelum = df.duplicated().sum()
8 print(f'Jumlah data duplikat sebelum dibersihkan: {jumlah_duplikat_sebelum}")
9
10 # Membersihkan data duplikat
11 df_cleaned = df.drop_duplicates()
12
13 # Menampilkan jumlah data duplikat setelah dibersihkan
14 jumlah_duplikat_setelah = df_cleaned.duplicated().sum()
15 print(f'Jumlah data duplikat setelah dibersihkan: {jumlah_duplikat_setelah}")
16
17 # Menyimpan data bersih ke file CSV
18 df_cleaned.to_csv('data_cleaned2.csv', index=False)
19
20 # Dataframe df_cleaned sekarang berisi data tanpa duplikat
```

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: 'should_run_async' will
and should_run_async(code)
```

Jumlah data duplikat sebelum dibersihkan: 5163
Jumlah data duplikat setelah dibersihkan: 0

Gambar 3.4

Selanjutnya memeriksa duplikasi data. Dilakukan pengecekan data duplikat dan didapatkan total data duplikat dari dataset yaitu 5163 yang kemudian data-data tersebut dibersihkan hingga jumlah data duplikat menjadi 0. Pengecekan dan pembersihan data duplikat dilakukan dengan baris kode seperti pada Gambar 3.4.

- Memeriksa kembali null values

```
[ ] 1 import pandas as pd
2
3 # Baca data dari file CSV
4 df = pd.read_csv('data_cleaned2.csv')
5
6 # Cetak jumlah nilai null
7 print(df.isnull().sum())

/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: "should_run_async" will
and should_run_async(code)
TransactionNo    0
Date            0
ProductNo       0
ProductName      0
Price           0
Quantity        0
CustomerNo      1
Country         0
dtype: int64
```

Gambar 3.5

Dilakukan kembali pengecekan *null values* pada dataset, untuk memeriksa apakah data yang memiliki *null values* tersebut sudah dibersihkan (termasuk data duplikat) atau tidak. Setelah dilakukan pengecekan, ternyata *null values* pada kolom nomor pelanggan masih ada. Sehingga, perlu dilakukan pengecekan lebih terperinci untuk kolom nomor pelanggan.

- Investigasi baris data valid pada kondisi customer_no bernilai null

```
[ ] 1 import pandas as pd
2
3 # Membaca dataset CSV
4 df = pd.read_csv('data_cleaned2.csv')
5
6 # Filtering data untuk baris dengan CustomerNo yang bernilai null
7 filtered_df = df[df['CustomerNo'].isnull()]
8
9 # Menampilkan kolom TransactionNo, ProductName, dan CustomerNo (termasuk yang bernilai null)
10 result = filtered_df[['TransactionNo', 'ProductName', 'CustomerNo']]
11
12 # Menampilkan hasil
13 print(result)

/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: "should_run_async" will not
and should_run_async(code)
TransactionNo    ProductName    CustomerNo
290726    558245    Set Of 6 Ribbons Vintage Christmas    NaN
```

Gambar 3.6

Dilakukan investigasi baris data valid pada kondisi kolom nomor pelanggan (customer no) bernilai *null*. Didapatkan nilai *null* pada baris 290726. Selanjutnya dilakukan pembersihan terhadap data dengan nilai *null*. Dilakukan kembali pengecekan *null values* dan didapat bahwa *null values* sudah tidak ada di dalam dataset yang digunakan.

	transaction_no	date	product_no	product_name	price	quantity	customer_no	country
0	581482	2019-12-09	22485	Set Of 2 Wooden Market Crates	21.47	12	17490.0	United Kingdom
1	581475	2019-12-09	22596	Christmas Star Wish List Chalkboard	10.65	36	13069.0	United Kingdom
2	581475	2019-12-09	23235	Storage Tin Vintage Leaf	11.53	12	13069.0	United Kingdom
3	581475	2019-12-09	23272	Tree T-Light Holder Willie Winkie	10.65	12	13069.0	United Kingdom
4	581475	2019-12-09	23239	Set Of 4 Knock Knock Tins Poppies	11.94	6	13069.0	United Kingdom
...	...	...	...	...	...	...	...	...
522597	536585	2018-12-01	37449	Ceramic Cake Stand + Hanging Cakes	20.45	2	17460.0	United Kingdom
522598	536590	2018-12-01	22776	Sweetheart 3 Tier Cake Stand	20.45	1	13065.0	United Kingdom
522599	536590	2018-12-01	22622	Box Of Vintage Alphabet Blocks	20.45	2	13065.0	United Kingdom
522600	536591	2018-12-01	37449	Ceramic Cake Stand + Hanging Cakes	20.45	1	14605.0	United Kingdom
522601	536597	2018-12-01	22220	Cake Stand Lovelied 2 Tier White	20.45	1	18011.0	United Kingdom

Gambar 3.7

Memanggil kembali *data frame* untuk menampilkan dataset terbaru setelah dilakukan *data cleansing*, didapatkan jumlah baris dan kolom yang terbaru yaitu 522602 baris dan 8 kolom. Nama-nama kolom juga diubah mengikuti standar baku.

```
[ ] 1 import pandas as pd
2
3 # Baca dataset
4 dataset = pd.read_csv('data_cleaned2.csv')
5
6 # Hitung jumlah transaksi
7 jumlah_transaksi = len(dataset['transaction_no'].unique())
8
9 # Hitung jumlah pelanggan unik
10 jumlah_pelanggan = len(dataset['customer_no'].unique())
11
12 # Hitung jumlah produk unik
13 jumlah_produk = len(dataset['product_no'].unique())
14
15 # Output hasil
16 print(f"Jumlah Transaksi: {jumlah_transaksi}")
17 print(f"Jumlah Pelanggan Unik: {jumlah_pelanggan}")
18 print(f"Jumlah Produk Unik: {jumlah_produk}")

/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: "should_run_async" will
and should_run_async(code)
Jumlah Transaksi: 19790
Jumlah Pelanggan Unik: 4719
Jumlah Produk Unik: 3753
```

Gambar 3.8

Dari total dataset sepanjang 522602 baris, terdapat 4719 orang pelanggan, dengan jumlah transaksi dari total pelanggan tersebut adalah 19790 kali transaksi dengan jenis produk yang terjual yaitu sebanyak 3753 barang. Dari data tersebut dapat diartikan, selama 1 tahun 8 hari toko *online* tersebut berjalan, mereka sudah memiliki 4719 orang pelanggan yang melakukan pembelian di toko tersebut. Dengan total transaksi sebanyak 19790 kali melebihi pelanggan yang berbelanja kepada mereka, mengartikan bahwa pelanggan melakukan transaksi berulang di toko *online* tersebut.

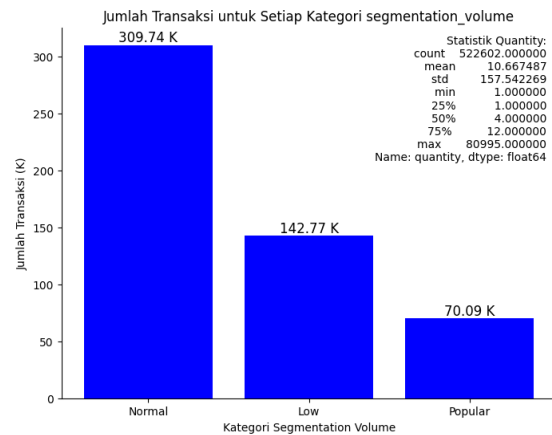
A. Produk dengan Kontribusi Tertinggi

Segmentasi volume barang (kuantitas) dilakukan untuk mengetahui segmentasi transaksi dari toko tersebut berdasarkan kuantitas produk yang dibeli oleh pelanggan per satu kali transaksi.

```
[ ] 1 import pandas as pd
2 import random
3
4 # Membaca dataset
5 data = pd.read_csv('data_cleaned2.csv')
6
7 # Membuat kolom total_revenue
8 data['total_revenue'] = data['price'] * data['quantity']
9
10 # Menghitung batas bin secara manual
11 bin_edges = [0, data['quantity'].quantile(0.2), data['quantity'].quantile(0.8), data['quantity'].max()]
12
13 # Mendefinisikan label kategori
14 segment_labels = ['Low', 'Normal', 'Popular']
15
16 # Membuat kolom segmentation_volume
17 data['segmentation_volume'] = pd.cut(data['quantity'], bins=bin_edges, labels=segment_labels, include_lowest=True)
18
19 # Menyimpan hasil program ke dalam segmented_vol.csv
20 data.to_csv('segmented_vol.csv', index=False)
21
22 # Menghitung jumlah transaksi dari setiap kategori
23 transaction_counts = data['segmentation_volume'].value_counts()
24
25 # Mencetak jumlah transaksi dari setiap kategori
26 print("Jumlah Transaksi untuk Setiap Kategori segmentation_volume :")
27 print(transaction_counts)
```

Gambar 3.9

Dari baris kode pada Gambar 3.9, dilakukan segmentasi volume barang dengan 3 tingkat, yaitu *low*, *normal*, dan *popular*. *Low* adalah golongan transaksi dengan tingkat kuantitas produk rendah, *normal* adalah golongan transaksi dengan tingkat kuantitas produk yang dibeli pada tingkat menengah atau normal, dan *popular* adalah golongan transaksi dengan tingkat kuantitas produk yang dibeli tinggi. Didapatkan hasil segmentasi berdasarkan kuantitas yaitu *low*: 142771 transaksi, *normal*: 309741 transaksi, dan *popular*: 70090 transaksi sebagaimana terlampir pada Grafik 3.1.



Grafik 3.1

Dari Grafik 3.1, dapat diinterpretasikan secara visual bahwa transaksi dengan kuantitas barang yang termasuk kategori normal adalah yang paling tinggi. Kategori kuantitas barang rendah jauh lebih rendah daripada kategori normal, yang berarti pelanggan memiliki kepercayaan dan hubungan yang baik untuk melakukan transaksi dengan kuantitas barang yang cukup banyak.

Selain segmentasi volume barang, dilakukan juga segmentasi berdasarkan total transaksi (*revenue*) untuk mengetahui kategori transaksi berdasarkan total belanja pelanggan.

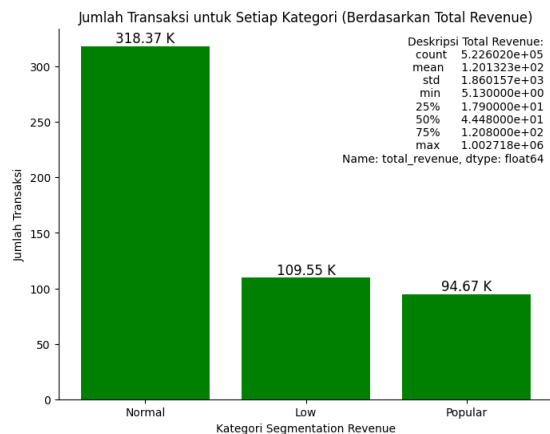
```
• Segmentasi Total Transaksi (Revenue)

1 # Menghitung persentil untuk segmentasi berdasarkan total revenue
2 quantiles = [0, 0.2, 0.8, 1]
3 segment_labels = ['Low', 'Normal', 'Popular']
4
5 # Membuat kolom total_revenue
6 data['total_revenue'] = data['price'] * data['quantity']
7
8 # Membuat kolom segmented_rev berdasarkan total revenue
9 data['segmented_rev'] = pd.cut(data['total_revenue'], data['total_revenue'].quantile(quantiles), labels=segment_labels)
10
11 # Menyimpan hasil program ke dalam segmented_rev.csv
12 data.to_csv('segmented_rev.csv', index=False)
13
14 # Menghitung jumlah transaksi dari setiap kategori
15 transaction_counts = data['segmented_rev'].value_counts()
16
17 # Mencetak jumlah transaksi dari setiap kategori
18 print("Jumlah Transaksi untuk Setiap Kategori:")
19 print(transaction_counts)
20
```

Gambar 3.10

Dari baris kode pada Gambar 3.10, transaksi dibagi ke dalam 3 kategori seperti pada segmentasi volume yaitu *low*, *normal*, dan *popular*. Hasil segmentasi berdasarkan *revenue* yaitu *low*: 109548 transaksi, *normal*: 318369 transaksi, dan

popular: 94670 transaksi dengan tampilan grafik seperti pada Grafik 3.2.



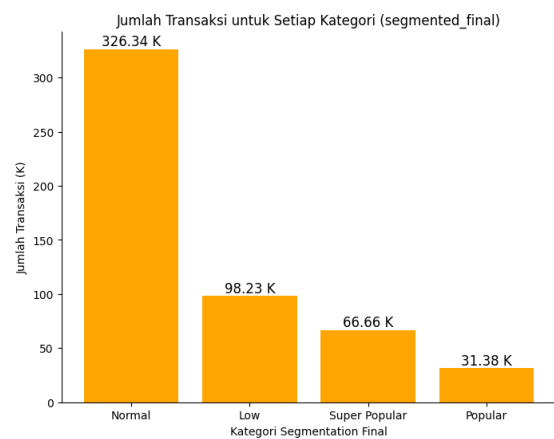
Grafik 3.2

Setelah melakukan segmentasi volume barang dan segmentasi total transaksi, kedua segmentasi tersebut digabungkan untuk mendapatkan informasi lebih lanjut.

```
1 data = pd.read_csv('segmented_rev.csv')
2
3 # Menggabungkan kolom 'segmentation_volume' dan 'segmented_rev' dengan tanda '-'
4 data['segmented_combine'] = data['segmentation_volume'].astype(str) + '-' + data['segmented_rev'].astype(str)
5
6 # Mengaturnya sesuai dengan aturan yang telah Anda tentukan
7 def determine_segmented_final(row):
8     if row['segmented_combine'] == 'Popular-Popular':
9         return 'Super Popular'
10    elif row['segmented_combine'] == 'Popular-Normal' or row['segmented_combine'] == 'Normal-Popular':
11        return 'Popular'
12    elif row['segmented_combine'] == 'Low-Low':
13        return 'Low'
14    return 'Normal'
15
16 data['segmented_final'] = data.apply(determine_segmented_final, axis=1)
17
18 # Menyimpan hasil program ke dalam segmented_final.csv
19 data.to_csv('segmented_final.csv', index=False)
20
21 # Menghitung jumlah produk dalam masing-masing kategori di kolom segmented_final
22 # Ini menghitung frekuensi kemunculan setiap nilai unik dalam kolom 'segmented_final.'
23 product_counts = data['segmented_final'].value_counts()
24
25 # Cetak hasilnya
26 print("Jumlah (count) Produk ditransaksikan dalam Setiap Kategori (segmented_final):")
27 print(product_counts)
```

Gambar 3.11

Baris kode pada Gambar 3.11 adalah baris kode yang menggabungkan kedua segmentasi tersebut. Segmentasi final ini terdiri dari 4 kategori yaitu *low*, *normal*, *popular*, dan *super popular*. Didapatkan hasil segmentasi yaitu *low*: 98229 transaksi, *normal*: 326338 transaksi, *popular*: 31378 transaksi, dan *super popular*: 66657 transaksi. Tampilan segmentasi final dalam bentuk grafik terlampir pada Grafik 3.3.



Grafik 3.3

B. Elastisitas Harga

Analisis yang umum dilakukan pada pasar ekonomi adalah elastisitas harga suatu barang. Dengan data yang dimiliki di dataset serta data-data yang sudah diolah sebelumnya, dapat dilakukan analisis elastisitas harga barang di toko *online* ini.

```
1 # Membaca dataset
2 df = pd.read_csv('segmented_final.csv')
3
4 # Mengelompokkan data berdasarkan product_no dan menghitung jumlah harga unik
5 unique_prices = df.groupby('product_no')['price'].nunique()
6
7 # Menyarung produk dengan setidaknya 10 variasi harga yang berbeda
8 products_with_10_or_more_prices = unique_prices[unique_prices >= 10].index
9
10 # Menampilkan produk yang memenuhi kriteria
11 print("Produk yang paling sering mengalami perubahan harga (setidaknya 10 variasi harga yang berbeda):")
12 print(products_with_10_or_more_prices)
13
14 Produk yang paling sering mengalami perubahan harga (setidaknya 10 variasi harga yang berbeda):
15 Index(['18113', '18036', '15039', '15850N', '16218', '16219', '16225', '17083',
16        '20815', '20819',
17        '85158', '85153', '85160A', '85169C', '85170D', '85173', '85174',
18        '85179A', '85179C', '851905'],
19       dtype='object', name='product_no', length=700)
```

Gambar 3.12

Baris kode pada Gambar 3.12 adalah pengelompokkan produk berdasarkan perubahan harga minimal 10 variasi atau lebih. Didapatkan 700 produk dengan variasi harga sebanyak 10 atau lebih. Artinya, dalam kurun waktu 1 tahun 8 hari, sekitar 700 produk di toko *online* ini mengalami perubahan harga.

```

1 df = pd.read_csv('segmented_final.csv')
2
3 # Mengelompokkan data berdasarkan product_no dan menghitung jumlah harga unik
4 unique_prices = df.groupby('product_no')['price'].nunique().reset_index()
5
6 # Menyaring produk dengan setidaknya 10 variasi harga yang berbeda
7 products_with_10_or_more_prices = unique_prices[unique_prices['price'] >= 10]
8
9 # Menggabungkan dengan data asli untuk mendapatkan nama produk
10 result = pd.merge(products_with_10_or_more_prices, df[['product_no', 'product_name']], on='product_no', how='inner')
11
12 # Mengelompokkan data berdasarkan product_no dan menghitung berapa kali harga berubah
13 count_diff_price = df.groupby('product_no').size().reset_index(name='count_diff_price')
14
15 # Menggabungkan jumlah perubahan harga ke dalam hasil
16 result = pd.merge(result, count_diff_price, on='product_no', how='inner')
17
18 # Menghilangkan duplikasi hasil
19 result = result.drop_duplicates(subset='product_no')
20
21 # Mengurutkan hasil berdasarkan count_diff_price dalam urutan menurun
22 result = result.sort_values(by='count_diff_price', ascending=False).head(10)
23
24 # Menampilkan 10 produk dengan perubahan harga terbanyak
25 print(result[['product_no', 'product_name', 'count_diff_price']])

```

Gambar 3.13

Di antara 700 barang yang mengalami perubahan harga dalam kurun waktu 1 tahun 8 hari tersebut, hendak diketahui 10 barang dengan perubahan harga terbanyak. 10 barang tersebut antara lain: 'Cream Hanging Heart T-Light Holder', 'Jumbo Bag Red Retrospot', 'Regency Cakestand 3 Tier', 'Party Bunting', 'Lunch Bag Red Retrospot', 'Assorted Colour Bird Ornament', 'Popcorn Holder', 'Set Of 3 Cake Tins Pantry Design', 'Pack Of 72 Retrospot Cake Cases', dan 'Lunch Bag Suki Design'.

```

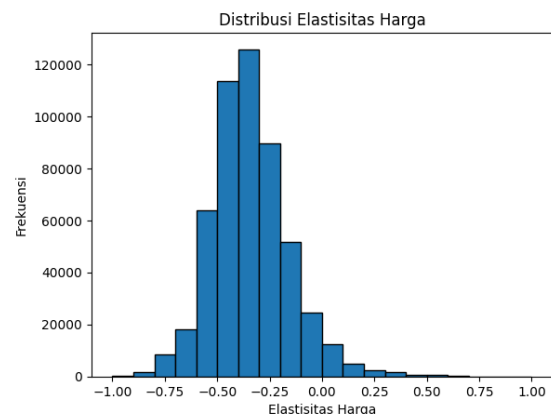
1 df = pd.read_csv('segmented_final.csv')
2
3 # Mengelompokkan data berdasarkan product_no
4 grouped = df.groupby('product_no')
5
6 # Inisialisasi list untuk menyimpan hasil korelasi
7 correlations = []
8
9 # Mengatur peringatan untuk diabaikan
10 warnings.filterwarnings("ignore")
11
12 # Menghitung korelasi antara harga dan kuantitas untuk setiap produk
13 for name, group in grouped:
14     if len(group) >= 2: # Hanya hitung korelasi jika ada dua atau lebih entri
15         correlation = group['price'].corr(group['quantity'], method='spearman')
16         correlations.append((name, correlation))
17
18 # Mengembalikan pengaturan peringatan ke keadaan semula
19 warnings.resetwarnings()
20
21 # Menambahkan kolom korelasi ke dataset
22 correlation_dict = dict(correlations)
23 df['korelasi'] = df[['product_no']].map(correlation_dict)
24
25 # Simpan dataset dengan kolom korelasi ke file CSV
26 df.to_csv('price_elasticity.csv', index=False)

```

Gambar 3.14

Baris kode pada Gambar 3.14 untuk melakukan uji korelasi antara harga dan kuantitas. Uji korelasi dilakukan dengan 3 jenis uji yaitu uji *Spearman*, uji *Pearson*, dan uji *Kendall*. Didapatkan hasil uji korelasi *Pearson* antara harga dan kuantitas adalah -0.0117; hasil uji korelasi *Spearman* antara harga dan kuantitas barang yaitu -0.3125; dan hasil uji korelasi

Kendall antara harga dan kuantitas barang adalah -0.2327.



Grafik 3.4

Grafik 3.4 menampilkan distribusi elastisitas harga barang-barang di toko *online* secara barchart. Modus dari nilai elastisitas harga produk di toko ini adalah pada interval -0.5 sampai dengan -0.25. Karena nilai elastisitas kurang dari 1 dan tidak sama dengan 0, maka harga barang-barang di toko sebagian besar bersifat *in elastis*. Dari grafik, dapat dilihat bahwa terdapat barang yang memiliki nilai elastisitas sama dengan 0, artinya barang-barang tersebut bersifat *in elastis* sempurna.

```

1 df = pd.read_csv('price_elasticity.csv')
2
3 # Mengelompokkan data berdasarkan product_no
4 grouped = df.groupby('product_no')
5
6 # Inisialisasi list untuk menyimpan hasil korelasi
7 correlations = []
8
9 # Mengatur peringatan untuk diabaikan
10 warnings.filterwarnings("ignore")
11
12 # Menghitung korelasi antara harga dan kuantitas untuk setiap produk
13 for name, group in grouped:
14     if len(group) >= 2: # Hanya hitung korelasi jika ada dua atau lebih entri
15         correlation = group['price'].corr(group['quantity'], method='spearman')
16         correlations.append((name, correlation))
17
18 # Mengembalikan pengaturan peringatan ke keadaan semula
19 warnings.resetwarnings()
20
21 # Menggunakan loop dan if statement untuk mengurutkan hasil korelasi dari yang paling negatif hingga yang paling positif
22 negative_correlations = [x for x in correlations if not pd.isna(x[1]) and x[1] < 0]
23 negative_correlations_sorted = sorted(negative_correlations, key=lambda x: x[1], reverse=True)
24
25 # Menampilkan 10 produk dengan korelasi negatif terbesar yang sudah diurutkan
26 top_10_negative_correlations = negative_correlations_sorted[:10]
27
28 print("\n10 Produk dengan Korelasi Paling Negatif:")
29 for i, (product_no, correlation) in enumerate(top_10_negative_correlations):
30     product_name = df[df['product_no'] == product_no]['product_name'].values[0]
31     print(f"{i+1}. Product No: {product_no} | Product Name: {product_name} | Korelasi: {correlation:.4f}")
32
33 # Menggunakan loop dan if statement untuk mengurutkan hasil korelasi dari yang paling positif hingga yang paling negatif
34 positive_correlations = [x for x in correlations if not pd.isna(x[1]) and x[1] > 0]
35 positive_correlations_sorted = sorted(positive_correlations, key=lambda x: x[1], reverse=True)
36
37 # Menampilkan 10 produk dengan korelasi positif terbesar yang sudah diurutkan
38 top_10_positive_correlations = positive_correlations_sorted[:10]
39
40 print("\n10 Produk dengan Korelasi Paling Positif:")
41 for i, (product_no, correlation) in enumerate(top_10_positive_correlations):
42     product_name = df[df['product_no'] == product_no]['product_name'].values[0]
43     print(f"{i+1}. Product No: {product_no} | Product Name: {product_name} | Korelasi: {correlation:.4f}")

```

Gambar 3.15

Gambar 3.15 adalah baris kode untuk mengetahui barang-barang dengan

tingkat korelasi paling positif dan paling negatif. Didapatkan hasil bahwa nilai korelasi paling positif yaitu 1 dan nilai korelasi paling negatif yaitu -1. Berikut daftar 10 barang dengan korelasi paling positif dan 10 barang dengan korelasi paling negatif pada Gambar 3.16.

```

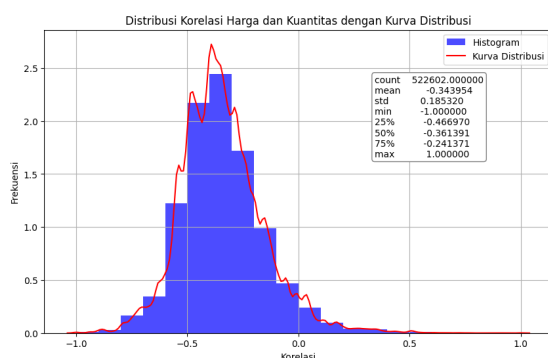
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: 'should_run_async'
and should_run_async(code)
10 Produk dengan Korelasi Paling Negatif:
1. Product No: 16849 | Product Name: Teatime Gel Pens Asst | Korelasi: -1.0000
2. Product No: 22827 | Product Name: Rustic Seventeen Drawer Sideboard | Korelasi: -1.0000
3. Product No: 22828 | Product Name: Regency Mirror With Shutters | Korelasi: -1.0000
4. Product No: 23441 | Product Name: Hand Painted Hanging Easter Egg | Korelasi: -1.0000
5. Product No: 35443 | Product Name: Rasta In Bath W Spillf Ashtray | Korelasi: -1.0000
6. Product No: 35618A | Product Name: Pink Christmas Flock Droplet | Korelasi: -1.0000
7. Product No: 35809B | Product Name: Enamel Blue Rim Tea Container | Korelasi: -1.0000
8. Product No: 84415B | Product Name: Blue Flock Cushion Cover | Korelasi: -1.0000
9. Product No: 84691 | Product Name: Pack 20 Dolly Pegs | Korelasi: -1.0000
10. Product No: 85067 | Product Name: Cream Sweetheart Wall Cabinet | Korelasi: -1.0000

10 Produk dengan Korelasi Paling Positif:
1. Product No: 98061A | Product Name: Antique Raspberry Flower Earrings | Korelasi: 1.0000
2. Product No: 98061B | Product Name: Antique Mid Blue Flower Earrings | Korelasi: 1.0000
3. Product No: 98038C | Product Name: Glass And Beads Bracelet Ivory | Korelasi: 1.0000
4. Product No: 98128A | Product Name: Pink Murano Twist Bracelet | Korelasi: 1.0000
5. Product No: 37351 | Product Name: Orange Flower Rug | Korelasi: 1.0000
6. Product No: 72783 | Product Name: Black Sil't Sou Candle Plate | Korelasi: 1.0000
7. Product No: 79157B | Product Name: Ubo-Light Triobase Blue | Korelasi: 1.0000
8. Product No: 84201B | Product Name: Happy Birthday Card Stripey Teddy | Korelasi: 1.0000
9. Product No: 84206C | Product Name: Champagne Tray Blank Card | Korelasi: 1.0000
10. Product No: 98176A | Product Name: Classic Diamante Necklace Jet | Korelasi: 1.0000

```

Gambar 3.16

Jumlah total produk dengan korelasi -1 adalah 32 dan korelasi 1 adalah 10.



Grafik 3.5

Grafik 3.5 adalah visualisasi dari distribusi korelasi harga dan kuantitas dengan kurva distribusi. Dapat dilihat bahwa Histogram dan Kurva distribusi memiliki garis lienar yang saling sejajar.

C. Analisis Time Series

Analisis time series dilaksanakan untuk mengetahui tren yang terjadi berdasarkan urutan waktu, seperti

pendapatan per hari, per minggu, per bulan, dan lain-lain.

```

1 df = pd.read_csv("segmented_final.csv")
2
3 # Mengubah kolom 'date' menjadi tipe data datetime
4 df['date'] = pd.to_datetime(df['date'])
5
6 # Mengelompokkan data berdasarkan tanggal dan menjumlahkan total penjualan per hari
7 daily_total_sales = df.groupby(['date'])['total_revenue'].sum().reset_index()
8
9 # Menampilkan hasil
10 print(daily_total_sales)

```

	date	total_revenue
0	2018-12-01	325557.05
1	2018-12-02	366528.50
2	2018-12-03	285865.11
3	2018-12-05	195080.25
4	2018-12-06	271379.16
...	...	...
380	2019-12-05	288765.67
381	2019-12-06	197034.00
382	2019-12-07	263857.24
383	2019-12-08	222840.66
384	2019-12-09	1106037.97

[385 rows x 2 columns]

Gambar 3.17

Gambar 3.17 adalah baris kode dan *output* dari prosedur pengelompokkan pendapatan berdasarkan hari. Dari hasil pengelompokkan tersebut, di dapat 305 baris untuk jangka waktu 1 tahun 8 hari.

```

1 df = pd.read_csv('segmented_final.csv')
2
3 # Mengurutkan dataset berdasarkan total_revenue secara descending
4 sorted_df = df.sort_values(by='total_revenue', ascending=False)
5
6 # Mengambil dua baris pertama
7 top_2_revenues = sorted_df.head(2)
8
9 # Memilih kolom yang ingin ditampilkan
10 selected_columns = ['product_name', 'quantity', 'price', 'total_revenue']
11
12 # Menampilkan hasil
13 result = top_2_revenues[selected_columns]
14 print(result)

```

	product_name	quantity	price	total_revenue
142	Paper Craft Little Birdie	80995	12.38	1002718.1
461881	Medium Ceramic Top Storage Jar	74215	11.32	840113.8

Gambar 3.18

Baris kode dan *output* pada Gambar 3.18 adalah baris kode program untuk mengetahui 2 produk dengan total *revenue* terbanyak. Produk pertama yaitu 'Paper Craft Little Birdie' dengan kuantitas 8099 yang dibandrol dengan harga 12.38 Dollar sehingga mendapatkan *revenue* sebesar 1002718.1. Produk kedua yaitu 'Medium Ceramic Top Storage Jar' dengan kuantitas 74215 dengan harga per barang yaitu 11.32 Dollar dengan total *revenue* yaitu 840113.8 Dollar.

1 monthly_total_sales		
	date	total_revenue
0	2018-12-31	4397648.39
1	2019-01-31	4548423.47
2	2019-02-28	3327342.64
3	2019-03-31	4384669.82
4	2019-04-30	3579310.06
5	2019-05-31	4569952.21
6	2019-06-30	4486132.15
7	2019-07-31	4571494.88
8	2019-08-31	4749801.23
9	2019-09-30	6613772.79
10	2019-10-31	7212279.85
11	2019-11-30	7828489.53
12	2019-12-31	2512069.52

Gambar 3.19

Gambar 3.19 menampilkan *summary* dari total pendapatan toko tersebut per bulan selama 1 tahun tersebut. Pendapatan toko paling tinggi yaitu pada bulan November 2019. Pendapatan toko paling rendah yaitu bulan Desember 2019.

```

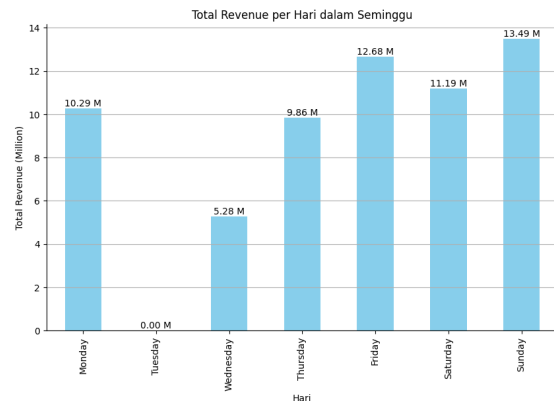
df = pd.read_csv('segmented_final.csv')
# Mengubah kolom 'date' menjadi tipe data datetime
df['date'] = pd.to_datetime(df['date'])
# Mengelompokkan data berdasarkan hari dalam seminggu (Senin hingga Minggu)
df['day_of_week'] = df['date'].dt.day_name()
# Mengelompokkan data dan menjumlahkan total penjualan per hari
df['total_sales_by_day'] = df.groupby('day_of_week')['total_revenue'].sum().reset_index(['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'])
# Mengganti nilai NaN dengan 0
df['total_sales_by_day'] = df['total_sales_by_day'].fillna(0)
# Menampilkan hasil
print(df['total_sales_by_day'])

```

Gambar 3.20

Gambar 3.20 merupakan baris kode dan *output* dari pengelompokkan pendapatan berdasarkan hari-hari dalam seminggu. Setelah dilakukan pengolahan, ternyata data pada hari Selasa tidak ada, menandakan bahwa toko mungkin tidak beroperasi pada hari Selasa. Pendapatan toko tertinggi yaitu pada hari Minggu, tertinggi kedua yaitu Jumat, dan ketiga Sabtu sebagaimana digambarkan pada Grafik 3.6. Hal ini menandakan bahwa pelanggan suka melakukan kegiatan berbelanja mereka pada *Weekend*. Untuk menjaga stabilitas pendapatan setiap harinya, toko bisa melakukan *offer* kepada

pelanggan berupa promo, potongan harga, dan lain-lain.



Grafik 3.6

IV. Kesimpulan

Berdasarkan hasil pengolahan dan analisis dari dataset transaksi pada bisnis toko online di Inggris dalam kurun waktu 373 hari yang terdiri dari 536.350 baris dan 8 kolom yang kemudian melalui data cleansing menjadi 522.602 baris dan 8 kolom, didapatkan beberapa poin kesimpulan sebagai berikut:

1. Terdapat 4719 orang pelanggan dengan jumlah total transaksi sebanyak 19.790 kali pada 3.753 barang jenis produk yang terjual. Hal ini menunjukkan bahwa pelanggan melakukan transaksi yang berulang di toko online tersebut.
2. Berdasarkan segmentasi volume barang yang dilakukan, terdapat 4 kategori dengan urutan dari jumlah transaksi terbanyak yakni Normal(326.338), Low(98.229), Super Popular(66.657) dan Popular(31.378)
3. Berdasar hasil uji korelasi(Metode Spearman, Pearson dan Kendall) antara harga dan kuantitas untuk mengetahui nilai

elastisitas harga, didapatkan hasil distribusi nilai modulus elastisitas dari rentang nilai -0,5 hingga -0,25 yang berarti nilai elastisitas kurang dari 1, Maka harga barang di toko tersebut sebagian besar bersifat in elastis.

4. Berdasarkan analisis data, diketahui produk dengan total revenue terbesar yakni "Paper Craft Little Birdie" dengan kuantitas 8099 dengan harga 12,38 Dollar sehingga revenue sebesar 1.002.718,1 Dollar.

5. Hasil analisis data menunjukkan pendapatan toko tertinggi pada bulan November 2019 dan paling rendah pada bulan Desember 2019 dengan hari beroperasi setiap hari kecuali hari selasa. Pendapatan tertinggi toko didapat pada hari Minggu, Jumat dan Sabtu yang menunjukkan bahwa pelanggan menyukai kegiatan berbelanja di akhir pekan.

Daftar Pustaka

[1] M. Danuri, "PERKEMBANGAN DAN TRANSFORMASI TEKNOLOGI DIGITAL," *INFOKAM*, no. 2, p. 117, 2019.

[2] O. and A. Agit, "Penerapan Penggunaan E-Payment Sebagai Upaya Peningkatan Adaptabilitas UMKM Terhadap Transformasi Digital," *Jurnal Pengabdian Kepada Masyarakat*, vol. 4, no. 2, p. 237, 2023.

[3] Y. A. Rohman, "Pengenalan NumPy, Pandas, Matplotlib," *Medium*, 18 Desember 2019. [Online]. Available: <https://medium.com/@yasirabd/pengenalan-numpy-pandas-matplotlib-b90bafd36c0>. [Accessed 2023].

[4] R. Fajar, "Menggunakan Library Random di Python," *CODEPOLITAN*, 19 Januari 2017. [Online]. Available: <https://www.codepolitan.com/blog/menggunakan-library-random-di-python-5879a3318b59d/>. [Accessed 2023].

[5] H. Wijoyo, "eprints.utdi," *BAB II TINJAUAN PUSTAKA DAN DASAR TEORI*, pp. 18-19.